# Multi-Asset Quanto Pricing via C++ and Python Interface *

Samson Lam Alex Wong

April 2026

## 1 Introduction

A quanto option is written on an asset quoted in one currency and pays in another at a fixed conversion rate. The holder receives the asset's performance in the payoff currency without bearing exchange-rate risk. This report extends qflib, a C++ library with a Python interface, to quanto options, and examines how the correlations between each asset and its exchange rate and among the assets affect their prices. European quanto options are priced by three methods, a closed-form formula, Monte Carlo simulation and a finite-difference scheme for the pricing partial differential equation (PDE), and quanto basket calls and quanto worst-of digitals by Monte Carlo. The correlation between the asset and the foreign exchange (FX) rate enters only through the drift, so a higher correlation raises call prices and lowers put prices. For baskets, the asset-asset correlation acts through dispersion instead. A basket call is cheaper than the weighted average of its component calls by the convexity of the payoff, and the gap narrows as the asset-asset correlation approaches one.

Section 2 derives the quanto drift, the closed-form price, put-call parity and the correlation sensitivity. Section 3 states the Monte Carlo and PDE schemes and the multi-asset extension. Section 4 describes the C++ design and the Python interface. Section 5 validates the single-asset pricers and studies the asset-FX correlation. Section 6 treats quanto baskets and worst-of digitals. Section 7 concludes.[1]

## 2 Model

The asset $S_t$ is quoted in currency CCY1, pays a continuous dividend yield $q$, and the option pays in currency CCY2. The continuously compounded interest rates of CCY1 and CCY2 are denoted as $r_f$ and $r_d$. The exchange rate $X_t$ is the number of units of CCY1 per unit of CCY2, and $Y_t = 1/X_t$ is the CCY2 price of one unit of CCY1. Under the CCY2 risk-neutral measure $\mathbb{Q}^d$,

$$\frac{dS_t}{S_t} = \mu\, dt + \sigma_S\, dW_t^S, \qquad \frac{dX_t}{X_t} = \mu_X\, dt + \sigma_X\, dW_t^X, \qquad d\langle W^S, W^X\rangle_t = \rho\, dt, \tag{1}$$

where $W^S$ and $W^X$ are $\mathbb{Q}^d$ Brownian motions, $\sigma_S$ and $\sigma_X$ are constant volatilities, and the drift $\mu$ is derived in Section 2.1. A quanto call ($\phi = 1$) or put ($\phi = -1$) with strike $K$ and expiry $T$ pays $(\phi(S_T - K))^+$ units of CCY2.

### 2.1 Quanto drift

One unit of CCY1 deposited at $r_f$ is a CCY2 asset worth $e^{r_f t} Y_t$, so $e^{(r_f - r_d)t} Y_t$ is a $\mathbb{Q}^d$ martingale (Garman and Kohlhagen, 1983), so

$$\frac{dY_t}{Y_t} = (r_d - r_f)\, dt - \sigma_X\, dW_t^X. \tag{2}$$

*This is a class project for Fordham University's QFGB 8960 course.

[1] Code is available at *GitHub*.

Holding one unit of the asset with dividends reinvested is worth $e^{qt}S_tY_t$ in CCY2, so the discounted process $G_t = e^{(q-r_d)t}S_tY_t$ is also a $\mathbb{Q}^d$ martingale,

$$\frac{dG_t}{G_t} = (q - r_d + \mu + r_d - r_f - \rho\,\sigma_S\sigma_X)\,dt + \sigma_S\,dW_t^S - \sigma_X\,dW_t^X. \tag{3}$$

Setting the drift of $G_t$ to zero and writing $\mu = r_d - q'$ yields the quanto drift and the equivalent dividend yield $q'$,

$$\mu = r_f - q + \rho\,\sigma_S\sigma_X, \qquad q' = r_d - r_f + q - \rho\,\sigma_S\sigma_X. \tag{4}$$

The quanto forward is therefore

$$F^Q = \mathbb{E}^{\mathbb{Q}^d}[S_T] = S_0\,e^{(r_f - q + \rho\,\sigma_S\sigma_X)T}. \tag{5}$$

A positive $\rho$ means the asset tends to rise when CCY1 weakens against CCY2. The CCY2 value $S_tY_t$ must still grow at $r_d - q$, so the negative covariance between $S_t$ and $Y_t$ is offset by the quanto adjustment $\rho\,\sigma_S\sigma_X$ in the drift of $S_t$. The quanto contract removes $Y_t$ from the payoff and keeps that adjustment.

### 2.2 Closed-form price

Under $\mathbb{Q}^d$, $S_T = F^Q \exp(-\frac{1}{2}\sigma_S^2 T + \sigma_S\sqrt{T}\,Z)$ with $Z$ standard normal. With $\Phi$ the standard normal distribution function and $\Phi'$ its density, the price is

$$V_\phi = \phi\,e^{-r_dT}\left[F^Q\Phi(\phi d_1) - K\Phi(\phi d_2)\right], \qquad d_{1,2} = \frac{\ln(F^Q/K) \pm \frac{1}{2}\sigma_S^2 T}{\sigma_S\sqrt{T}}, \tag{6}$$

which is the formula of Black and Scholes (1973), extended to a continuous dividend yield by Merton (1973), with interest rate $r_d$ and dividend yield $q'$.

### 2.3 Put-call parity

The prices in (6) satisfy put-call parity,

$$V_+ - V_- = e^{-r_dT}\left(F^Q - K\right). \tag{7}$$

As $K \to 0$, the put price vanishes, so the call price tends to the discounted quanto forward.

### 2.4 Correlation sensitivity

The correlation enters the quanto price (6) only through $F^Q$, with $\partial F^Q/\partial\rho = \sigma_S\sigma_X T\,F^Q$. Hence,

$$\begin{aligned}
\frac{\partial V_\phi}{\partial\rho} &= \phi\,\sigma_S\sigma_X T\,e^{-r_dT}F^Q\Phi(\phi d_1) \\
\frac{\partial^2 V_\phi}{\partial\rho^2} &= (\sigma_S\sigma_X T)^2 e^{-r_dT}F^Q\left[\phi\,\Phi(\phi d_1) + \frac{\Phi'(d_1)}{\sigma_S\sqrt{T}}\right].
\end{aligned} \tag{8}$$

The call price increases and the put price decreases in $\rho$, and the call is convex in $\rho$.

# 3 Numerical Methods

## 3.1 Monte Carlo

On fixing times $0 = t_0 < t_1 < \cdots < t_m$ the pricer advances the logarithm of the asset exactly,

$$S_{t_i} = S_{t_{i-1}} \exp\Big[\big(r_f(t_{i-1}, t_i) - q + \rho\,\sigma_S\sigma_X - \tfrac{1}{2}\sigma_S^2\big)\,\Delta t_i + \sigma_S\sqrt{\Delta t_i}\,\varepsilon_i\Big], \quad \varepsilon_i \overset{i.i.d.}{\sim} \mathcal{N}(0,1), \tag{9}$$

where $r_f(t_{i-1}, t_i)$ is the CCY1 forward rate, $\Delta t_i = t_i - t_{i-1}$.

## 3.2 PDE

By the Feynman–Kac theorem (Shreve, 2004, Chapter 6), the price $V(t, x)$ as a function of $x = \ln S$ solves

$$\frac{\partial V}{\partial t} + \big(\mu - \tfrac{1}{2}\sigma_S^2\big)\frac{\partial V}{\partial x} + \tfrac{1}{2}\sigma_S^2\frac{\partial^2 V}{\partial x^2} - r_d V = 0, \qquad V(T, x) = (\phi(e^x - K))^+. \tag{10}$$

The solver uses a uniform grid in $x$ that spans 5 standard deviations around the forward and passes through $\ln S_0$. With $\mathcal{L}$ the central difference approximation of the drift and diffusion terms and $\theta \in [0, 1]$, each backward step solves

$$(I - \theta\,\Delta t\,\mathcal{L})\,u^n = (I + (1-\theta)\,\Delta t\,\mathcal{L})\,V^{n+1}, \qquad V^n = e^{-r_d\Delta t}u^n, \tag{11}$$

where the drift in $\mathcal{L}$ is taken from the one-step forward factor $e^{\mu\Delta t}$. We use $\theta = \frac{1}{2}$, the scheme of Crank and Nicolson (1947), with $N$ time steps and $N$ spot nodes.

## 3.3 Multi-asset extension

Asset $j = 1, \ldots, M$ is quoted in its own currency with interest rate $r_j$, dividend yield $q_j$ and volatility $\sigma_j$, and its exchange rate $X_j$ has volatility $\sigma_{X_j}$ and correlation $\rho_j$ with the asset. Under $\mathbb{Q}^d$,

$$\frac{dS_{j,t}}{S_{j,t}} = \mu_j\,dt + \sigma_j\,dW_t^j, \qquad \mu_j = r_j - q_j + \rho_j\,\sigma_j\sigma_{X_j}, \qquad d\langle W^j, W^k\rangle_t = \rho_{jk}\,dt. \tag{12}$$

The drift of asset $j$ involves only its own exchange rate, and by the Girsanov theorem (Shreve, 2004, Chapter 5) a change of measure does not alter instantaneous correlations. The joint law of the asset prices under $\mathbb{Q}^d$ therefore depends on $\mu_j$, $\sigma_j$ and the correlation matrix $R = (\rho_{jk})$ alone, and correlations between exchange rates or between an asset and another asset's exchange rate do not enter.

Two products are priced under (12).

1. With weights $w_j > 0$, the quanto basket option pays $(\phi(B_T - K))^+$ units of CCY2, where $B_T = \sum_j w_j S_{j,T}$ and $K$ is in CCY2.
2. The worst-of digital call with fixing time $T_1$, expiry $T_2$ and strike ratio $\kappa$ pays one unit of CCY2 when $\min_j S_{j,T_2}/S_{j,T_1} \geq \kappa$, and the put pays one unit otherwise.

# 4 Implementation via C++ and Python Interface

qflib is a C++20 library whose vectors and matrices are the Armadillo types. It is compiled into a static library and exposed to Python through an extension module written against the C application programming interfaces (APIs) of Python and NumPy. Figure 1 shows the layers and marks the

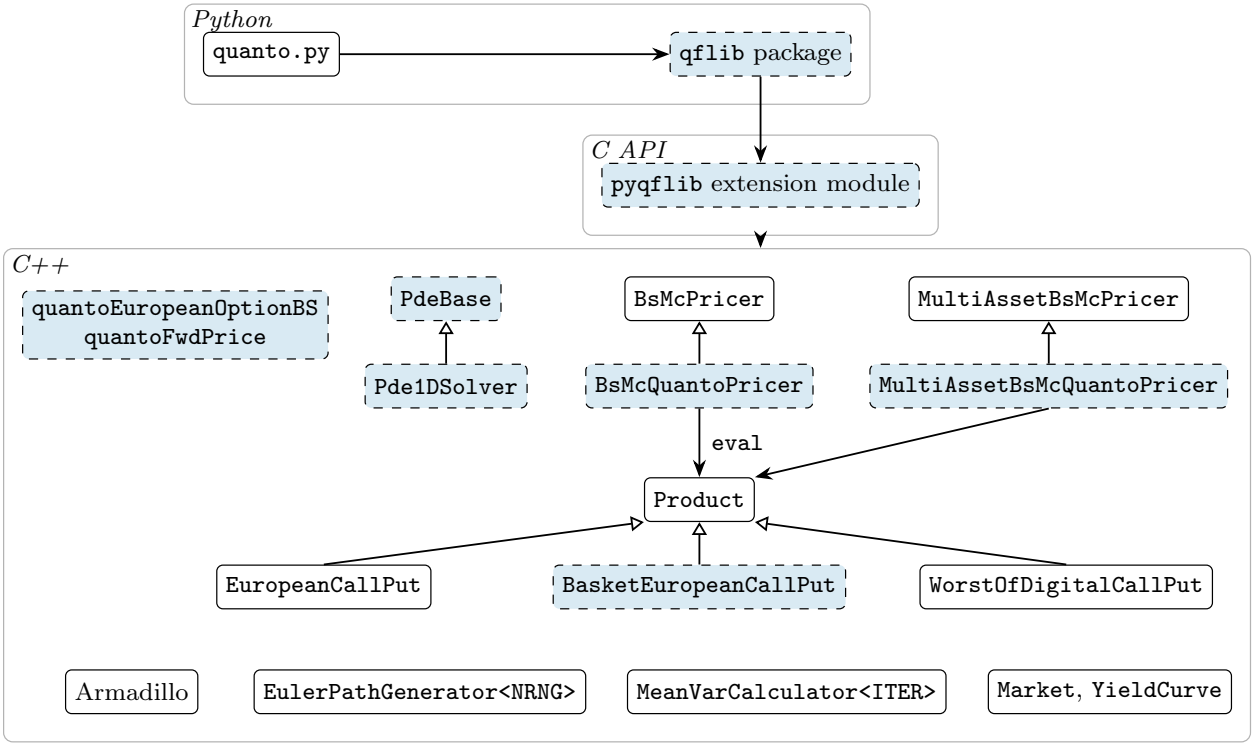



Figure 1: Architecture of qflib.

| Python function | Method | Pricer | Payoff |
|---|---|---|---|
| `qFwdPrice` | Closed form | `quantoFwdPrice` | |
| `qEuroBS` | Closed form | `quantoEuropeanOptionBS` | |
| `qEuroBSMC` | Monte Carlo | `BsMcQuantoPricer` | `EuropeanCallPut` |
| `qEuroBSPDE` | PDE | `Pde1DSolver` | |
| `qBasketEuroBSMC` | Monte Carlo | `MultiAssetBsMcQuantoPricer` | `BasketEuropeanCallPut` |
| `qWorstofDigiBSMC` | | | `WorstOfDigitalCallPut` |

Table 1: Quanto functions.

new classes in shaded dashed boxes, and Table 1 lists the new Python functions, with code excerpts in Listings 1 to 3 of the appendix.

CMake builds `qflib` as a static library and `pyqflib` as a shared library, a `.pyd` file on Windows, linked against `qflib`, the LAPACK and BLAS libraries shipped with Armadillo and the Python library. The package is installed with `pip`, and the script `quanto.py` produces the results of Sections 5 and 6 by calling the C++ pricers from Python.

## 5 Single-Asset Results

Table 2 lists the inputs. Both yield curves are flat, with USD as the payoff currency and EUR as the asset currency.

| Single asset | | Basket and worst-of | |
|---|---|---|---|
| $S_0$, $K$ | 100 | $S_{j,0}$, $w_j$ | 100, 1/3 |
| $T$ | 1 year | $\sigma_j$ | 20%, 25%, 30% |
| $r_d$, $r_f$ | 5%, 4% | $r_j$, $q_j$, $\sigma_{X_j}$, $\rho_j$ | 4%, 2%, 10%, 0.3 |
| $q$ | 2% | $\rho_{jk}$ | 0.5 |
| $\sigma_S$, $\sigma_X$ | 20%, 10% | $K$ | 100 |
| $\rho$ | 0.3 | $\kappa$, $T_1$, $T_2$ | 1, 3 months, 1 year |
| Monte Carlo | 200,000 paths | | |
| PDE | $N = 200$, $\theta = 0.5$, 5 standard deviations | | |

Table 2: Model and numerical inputs.

The PDE prices satisfy put-call parity (7) to within $1.4 \times 10^{-11}$ at three expiries and three strikes (Table 3).

| $T$ | $K$ | Call | Put | Residual |
|---|---|---|---|---|
| 6 months | 80 | 21.03 | 0.24 | $1.3 \times 10^{-12}$ |
| | 100 | 6.19 | 4.92 | $1.6 \times 10^{-12}$ |
| | 120 | 0.86 | 19.09 | $1.9 \times 10^{-12}$ |
| 1 year | 80 | 22.41 | 0.88 | $1.9 \times 10^{-12}$ |
| | 100 | 8.99 | 6.49 | $1.9 \times 10^{-12}$ |
| | 120 | 2.61 | 19.13 | $2.0 \times 10^{-12}$ |
| 2 years | 80 | 24.97 | 2.05 | $1.4 \times 10^{-11}$ |
| | 100 | 13.03 | 8.20 | $1.4 \times 10^{-11}$ |
| | 120 | 6.02 | 19.28 | $1.4 \times 10^{-11}$ |

Table 3: Put-call parity of PDE prices. The residual is $|V_+ - V_- - e^{-r_d T}(F^Q - K)|$.

Figure 2 compares both numerical methods with the closed-form at-the-money call price. The PDE error falls from 0.14 at $N = 25$ to $1.5 \times 10^{-4}$ at $N = 800$ with a fitted log-log slope of $-1.99$, the second-order convergence expected for the Crank–Nicolson scheme with both $\Delta t$ and $\Delta x$ proportional to $1/N$. Every Monte Carlo estimate lies within 1.3 standard errors of the closed form, and at $10^6$ paths the error is 0.002 against a standard error of 0.014.

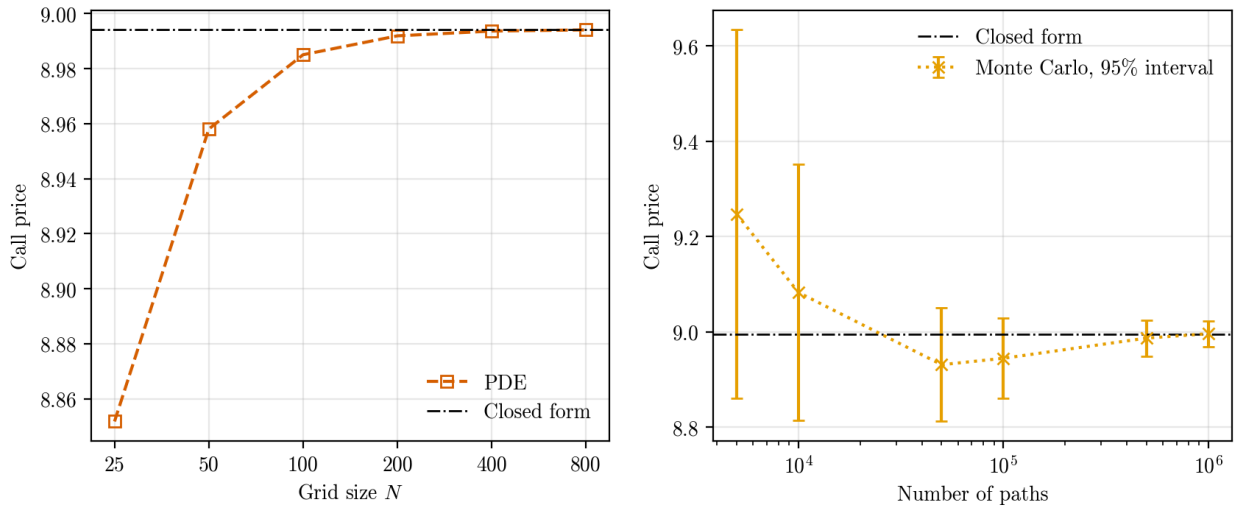
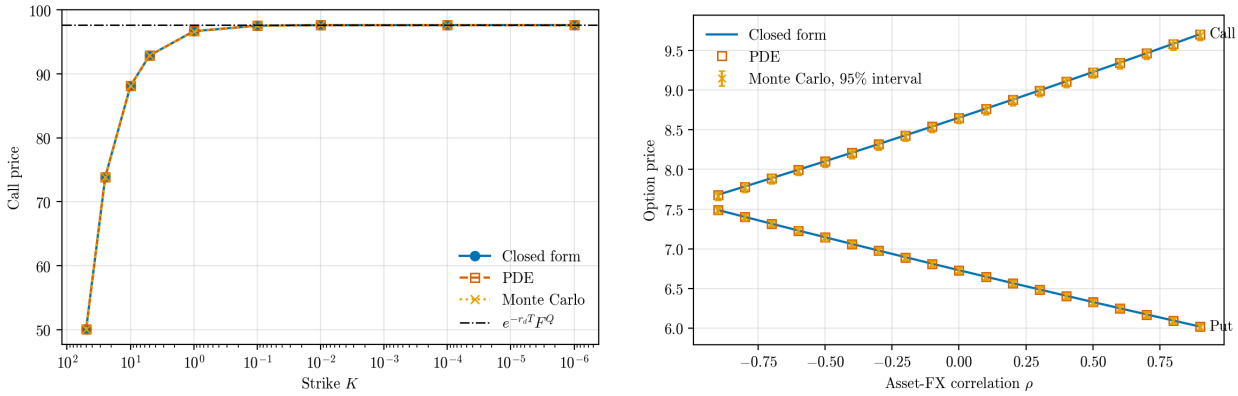



Figure 2: At-the-money call price against the PDE grid size and the number of Monte Carlo paths.

Figure 3(a) shows all three methods reaching the discounted quanto forward $e^{-r_d T} F^Q \approx 97.63$ as $K \to 0$. At $K = 10^{-6}$, the closed form and the PDE agree with the limit to 4 decimal places, and the Monte Carlo estimate of 97.62 lies 0.27 standard errors below it.

Figure 3(b) plots the at-the-money call and put prices for $\rho$ from $-0.9$ to 0.9. The call rises from 7.68 to 9.70, and the put falls from 7.49 to 6.02, as (8) predicts. Both curves are nearly linear because the second derivative in (8) carries the factor $4 \times 10^{-4}$. The PDE prices stay within $2.4 \times 10^{-3}$ of the closed form and the Monte Carlo prices within 0.5 standard errors.




(a) Call price as $K \to 0$.

(b) Call and put prices against $\rho$.

Figure 3: Single-asset quanto prices by the three methods.

# 6 Multi-Asset Results

## 6.1 Diversification

In Figure 4(a), the three component calls are worth 8.98, 10.96 and 12.94, their weighted average 10.96, and the basket call 9.23 for $\rho_{jk} = 0.5$. That the basket is cheaper follows from the convexity

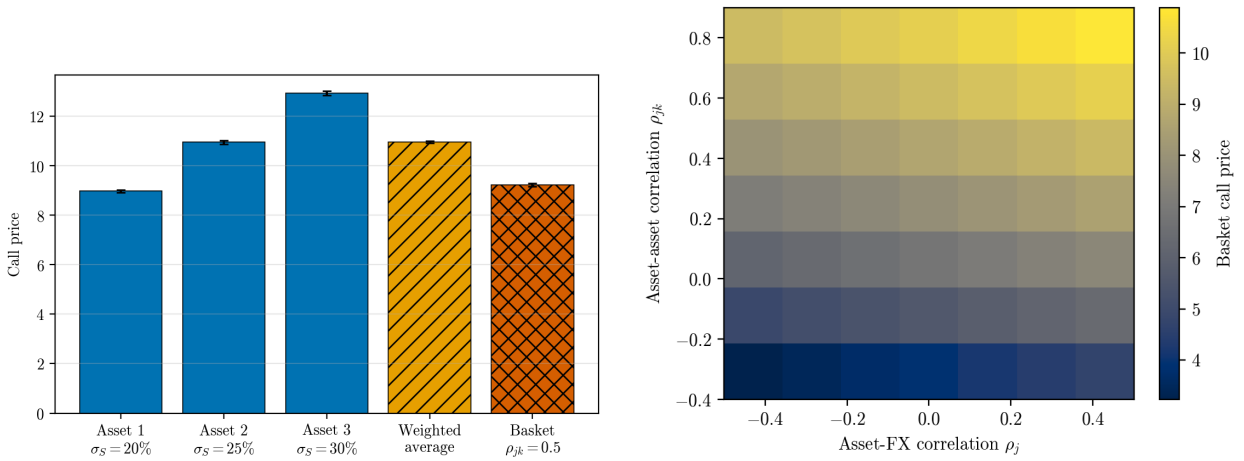
of the payoff. For $w_j \geq 0$ with $\sum_j w_j = 1$ and every realization, a call on a portfolio is worth no more than the portfolio of calls (Merton, 1973),

$$\Big(\sum_j w_j S_{j,T} - K\Big)^+ \leq \sum_j w_j \left(S_{j,T} - K\right)^+ . \tag{13}$$

The size of the gap depends on the basket volatility. Matching the first two moments of $B_T$ to those of a lognormal distribution (Levy, 1992) yields the basket volatility $\sigma_B$ with

$$\begin{aligned} \sigma_B^2 = \frac{1}{T} \ln \sum_{j,k} \omega_j \omega_k \, e^{\rho_{jk}\sigma_j\sigma_k T} \approx \sum_{j,k} \omega_j \omega_k \, \rho_{jk}\sigma_j\sigma_k \leq \Big(\sum_j \omega_j \sigma_j\Big)^2, \\ \omega_j = \frac{w_j F_j^Q}{\sum_k w_k F_k^Q}, \qquad F_j^Q = S_{j,0}\, e^{\mu_j T}. \end{aligned} \tag{14}$$

For an at-the-money-forward option ($K = F^Q$), (6) yields $V_+ \approx e^{-r_d T} F^Q \sigma_S \sqrt{T/2\pi}$, which increases and is nearly linear in volatility. At $\rho_{jk} = 0.5$ the basket volatility is 20.5% against an average component volatility of 25%, so the basket call at 9.23 lies much closer to the single 20% call at 8.98 than to the average of 10.96. This is the diversification effect, and it shrinks as $\rho_{jk}$ approaches one.




(a) Component calls and the basket call.

(b) Basket call against both correlations.

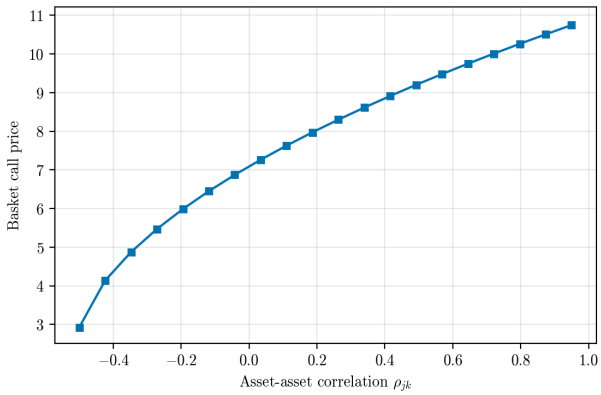
Figure 4: Quanto basket call prices. In panel (a) the error bars are 95% Monte Carlo intervals.

## 6.2 Asset-FX and asset-asset correlations

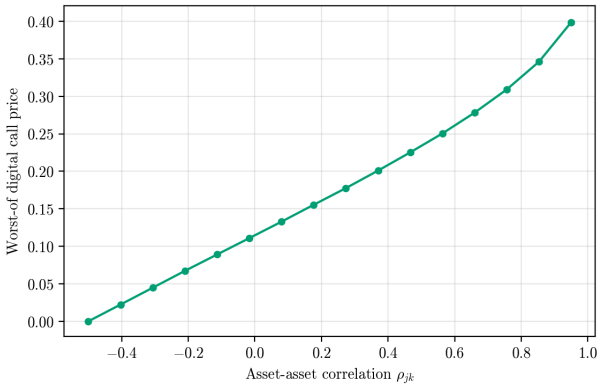
Figure 4(b) varies the common asset-FX correlation $\rho_j$ from $-0.5$ to 0.5 and $\rho_{jk}$ from $-0.4$ to 0.9. The basket call rises in both, from 3.23 to 10.88. The two correlations act through different moments. By (12), $\rho_j$ shifts each drift $\mu_j$ and hence each quanto forward, while by (14) $\rho_{jk}$ changes $\sigma_B$ alone.

## 6.3 Asset-asset correlation

Figure 5 varies a common $\rho_{jk}$ from $-0.5$, the smallest value for which $R$ is positive semidefinite, to 0.95. The basket call rises from 2.93 to 10.75 and approaches the weighted average of 10.96. The




(a) Basket call.




(b) Worst-of digital call.

Figure 5: Prices against a common asset-asset correlation.

curve is concave because $\sigma_B^2$ in (14) is approximately linear in $\rho_{jk}$, so $\sigma_B$ is concave in it, and the price is close to linear in $\sigma_B$.

The worst-of digital call rises from zero to 0.40. Since each $\ln(S_{j,T_2}/S_{j,T_1})$ is normal, the price is a joint normal probability,

$$V^{\mathrm{WO}} = e^{-r_d T_2}\,\mathbb{P}\left(\xi_j \geq c_j \text{ for all } j\right), \qquad \boldsymbol{\xi} \sim \mathcal{N}(\mathbf{0}, R), \qquad c_j = \frac{\ln\kappa - (\mu_j - \frac{1}{2}\sigma_j^2)\delta}{\sigma_j\sqrt{\delta}}, \tag{15}$$

where $\delta = T_2 - T_1$. This probability is nondecreasing in every $\rho_{jk}$ by the inequality of Slepian (1962), since the assets clear the strike together more often when they move together. Approximating the small thresholds $c_j$ by zero, write $A_j = \{\xi_j \geq 0\}$. For $M = 3$, the bivariate orthant probability of Sheppard (1899) yields

$$\mathbb{P}(A_j^c \cap A_k^c) = \frac{1}{4} + \frac{1}{2\pi}\arcsin\rho_{jk} \quad \Longrightarrow \quad \mathbb{P}\Big(\bigcap_j A_j\Big) = \frac{1}{8} + \frac{1}{4\pi}\sum_{j<k}\arcsin\rho_{jk}. \tag{16}$$

Hence, this approximation vanishes at $\rho_{jk} = -0.5$ and is convex for $\rho_{jk} > 0$, the shape of Figure 5(b).

## 7 Conclusion

This report examines how the asset-FX and asset-asset correlations affect quanto option prices, using closed-form, Monte Carlo and PDE pricers implemented in C++ with a Python interface. The three methods agree within their numerical error and recover the discounted quanto forward at zero strike, and the PDE prices satisfy put-call parity. The asset-FX correlation enters through the drift, raising calls and lowering puts. The multi-asset pricer needs no correlations between exchange rates for baskets in several currencies, because each asset's drift depends only on its own exchange rate. A quanto basket call is cheaper than the weighted average of its component calls by the convexity of the payoff, imperfect correlation widens the gap by lowering the basket volatility, and both basket calls and worst-of digital calls become more expensive as the assets become more correlated.

The model assumes constant volatilities and correlations, and extending the pricers to stochastic volatility is left for future research.

## References


Black, F. and Scholes, M., The pricing of options and corporate liabilities. *J. Polit. Econ.*, 1973, **81**(3), 637–654.

Crank, J. and Nicolson, P., A practical method for numerical evaluation of solutions of partial differential equations of the heat-conduction type. *Math. Proc. Camb. Philos. Soc.*, 1947, **43**(1), 50–67.

Garman, M.B. and Kohlhagen, S.W., Foreign currency option values. *J. Int. Money Financ.*, 1983, **2**(3), 231–237.

Levy, E., Pricing European average rate currency options. *J. Int. Money Financ.*, 1992, **11**(5), 474–491.

Merton, R.C., Theory of rational option pricing. *Bell J. Econ. Manag. Sci.*, 1973, **4**(1), 141–183.

Sheppard, W.F., On the application of the theory of error to cases of normal distribution and normal correlation. *Philos. Trans. R. Soc. Lond. Ser. A*, 1899, **192**, 101–167.

Shreve, S.E., *Stochastic Calculus for Finance II: Continuous-Time Models*, 2004 (Springer: New York).

Slepian, D., The one-sided barrier problem for Gaussian noise. *Bell Syst. Tech. J.*, 1962, **41**(2), 463–501.

# A Supplementary Materials

```
: MultiAssetBsMcPricer(prod, discountYieldCurve, divYields, volatilities, spots, correlMatrix, mcparams)
{
  size_t nassets = prod->nAssets();
  QF_ASSERT(growthYieldCurves.size() == nassets, "need as many growth yield curves as product assets!");
  QF_ASSERT(fxVols.size() == nassets, "need as many FX volatilities as product assets!");
  QF_ASSERT(assetFXCorrels.size() == nassets, "need as many asset-FX correlations as product assets!");

  // the base ctor sets the discount factors and stdevs;
  // the drifts are replaced by the quanto drifts on each asset currency curve
  Vector const& fixtimes = prod->fixTimes();
  for (size_t j = 0; j < nassets; ++j) {
    double t1 = 0.0;
    double quantoCoef = assetFXCorrels[j] * fxVols[j];
    for (size_t i = 0; i < fixtimes.size(); ++i) {
      double t2 = fixtimes[i];
      double var = vols_[j] * vols_[j] * (t2 - t1);
      double fwdrate = growthYieldCurves[j]->fwdRate(t1, t2);
      double quantoAdj = quantoCoef * vols_[j];
      drifts_(i, j) = (fwdrate - divylds_[j] + quantoAdj) * (t2 - t1) - 0.5 * var;
      t1 = t2;
    }
  }
}
```

Listing 1: Constructor body of `MultiAssetBsMcQuantoPricer`.

```
  QuantoAssetInputs in = asQuantoAssetInputs(pyDiscountCrv, pyGrowthCrvs, pyDivYields,
                                             pyVolatilities, pyFxVols, pyCorrels, pyCorrelMatrix);
  qf::McParams mcparams = asMcParams(pyMcParams);
  unsigned long npaths  = asInt(pyNPaths);

  // create the product
  qf::SPtrProduct spprod(new qf::BasketEuropeanCallPut(payoffType, strike, timeToExp, weights));
  // create the pricer
  qf::MultiAssetBsMcQuantoPricer bsmcpricer(spprod, in.discountCurve, in.growthCurves, in.divYields,
                                            in.vols, in.fxVols, in.correls, spots, in.correlMat, mcparams);
  // create the statistics calculator
  qf::MeanVarCalculator<double *> sc(bsmcpricer.nVariables());
  // run the simulation
  bsmcpricer.simulate(sc, npaths);
  // collect results
  qf::Matrix const & results = sc.results();
  // read out results
  double mean = results(0, 0);
  double stderror = results(1, 0);
  stderror = std::sqrt(stderror / sc.nSamples());

  // write mean and standard error into a Python dictionary
  PyObject* ret = PyDict_New();
  int ok = PyDict_SetItem(ret, asPyScalar("Mean"), asPyScalar(mean));
  PyDict_SetItem(ret, asPyScalar("StdErr"), asPyScalar(stderror));
  return ret;

PY_END;
```

Listing 2: Part of `pyQfBasketQEuroBSMC`, registered as `qBasketEuroBSMC`.

```
def basket_call(asset_corr, fx_correls=FX_CORRELS):
    return qf.qBasketEuroBSMC(CALL, K, T, spots=SPOTS.tolist(), weights=WEIGHTS.tolist(),
                              divyields=DIVS, volatilities=VOLS, fxvols=FX_VOLS, correls=fx_correls,
                              correlmatrix=uniform_corr(len(SPOTS), asset_corr), **basket_market(len(SPOTS))
    )
```

Listing 3: Basket call in `quanto.py`.